ECOLE MAROCAINE DES SCIENCES DE L'INGENIEUR
Membre de HONORIS UNITED UNIVERSITIES

PROJET DE FIN D'ANNÉE

« 3ème Année en Ingénierie Informatique et Réseaux »

Réalisé par :

Amargui Youssef, Bouras Aymane et Imhah Saad

Tuteur(s) :

Encadrant Pédagogique : Amina Ouatiq

Année universitaire : 2024/2025

Dédicaces

À l'issue de ce projet, nos pensées se tournent avec reconnaissance vers toutes les personnes qui ont contribué, de près ou de loin, à sa réalisation. À nos familles, pour leur soutien indéfectible, leur amour constant et les sacrifices souvent silencieux qu'elles ont consentis. À nos amis, dont la présence, l'écoute et les moments partagés nous ont apporté force et réconfort. À nos enseignants, pour la transmission de leur savoir, leur exigence empreinte de bienveillance et l'inspiration qu'ils nous ont donnée. À tous ceux qui ont croisé notre chemin, pour chaque mot, chaque geste, chaque aide, qu'elle ait été modeste ou précieuse. Ce projet est aussi le vôtre. Nous vous adressons notre profonde gratitude : merci.

Remerciements

Au terme de ce projet de fin d'études, nous souhaitons adresser nos plus vifs remerciements à celles et ceux qui ont rendu cette aventure possible.

Notre reconnaissance s'adresse tout particulièrement à **Madame Amina Ouatiq**, dont l'expertise et la bienveillance ont guidé chacun de nos pas. Son accompagnement rigoureux, sa patience inépuisable et ses précieux conseils ont été des atouts inestimables dans l'accomplissement de ce travail. Sa capacité à nous orienter tout en nous laissant développer notre autonomie a transformé ce projet en une expérience profondément formatrice.

Nous exprimons également notre gratitude envers **l'ensemble du corps professoral** qui a jalonné notre parcours académique. La richesse de leurs enseignements, leur passion communicative et leur exigence bienveillante ont forgé non seulement nos compétences techniques, mais aussi notre éthique professionnelle. Leurs observations pertinentes et leurs critiques constructives ont considérablement enrichi la qualité de notre réflexion et de notre production.

Que soient également remerciées **toutes les personnes** qui, dans l'ombre ou la lumière, ont apporté leur pierre à l'édifice de ce projet. Chaque contribution, aussi modeste soit-elle, a participé à l'aboutissement de ce travail dont nous sommes aujourd'hui fiers.

À tous, nous adressons l'expression de notre profonde gratitude.

Table des matières

Dédicaces **1**

Remerciements **2**

Introduction générale **4**

1 Présentation du cadre de projet **5**
1.1 Introduction 5
1.2 Contexte et état de l'art 5
1.2.1 Présentation des plateformes existantes 5
1.2.2 Limites des solutions actuelles 5
1.2.3 Problématique 5
1.3 Solution proposée 6
1.4 Choix de modèle de développement 6
1.5 Planning prévisionnel 6
1.6 Conclusion 6

2 Spécification des besoins **8**
2.1 Introduction 8
2.2 Spécification des besoins fonctionnels 8
2.2.1 Gestion des utilisateurs 8
2.2.2 Gestion de la musique (artistes, albums, titres) 8
2.2.3 Création et gestion des playlists 8
2.2.4 Système de recherche et recommandations 8
2.3 Spécification des besoins non fonctionnels 9
2.3.1 Interface utilisateur responsive et moderne 9
2.3.2 Compatibilité multiplateforme et navigateurs 9
2.3.3 Performances et optimisation du temps de chargement 9
2.3.4 Sécurité des données utilisateurs 9
2.3.5 Architecture modulaire et maintenabilité 9
2.3.6 Extensibilité et évolutivité 10
2.4 Cas d'utilisation 10
2.4.1 Acteurs du système 10
2.4.2 Description des cas d'utilisation 10
2.4.3 Diagramme de cas d'utilisation global 10

3 Conception du système **13**
3.1 Introduction 13
3.2 Modélisation dynamique 13
3.2.1 Diagrammes de séquences 13

3.2.2 Diagrammes de collaboration 13
3.2.3 Diagrammes d'états 14
3.2.4 Diagrammes d'état-transition 14
3.2.5 Diagrammes d'activité 15
3.3 Modélisation statique 15
3.3.1 Diagramme de classes 15
3.3.2 Modèle relationnel de la base de données 16
3.3.3 Architecture de l'application 16
3.4 Conclusion 17

4 Réalisation du système **18**
4.1 Introduction 18
4.2 Environnement de développement 18
4.2.1 Environnement matériel 18
4.2.2 Environnement logiciel 18
4.3 Principales interfaces graphiques 19
4.3.1 Page d'accueil 19
4.3.2 Page de création de compte 19
4.3.3 Page de connexion 19
4.3.4 Formulaire de création de playlist 20
4.3.5 Interface de lecture musicale 20
4.3.6 Interface de recherche 21
4.4 Conclusion 21

Conclusion générale **22**

Bibliographie et Nétographie **23**

ANNEXES **24**
ANNEXE A : Captures d'écran supplémentaires 24
ANNEXE B : Extraits de code ou scripts utiles 25
ANNEXE C : Recommandations et perspectives 26

Table des figures

1.1 Le cycle en V appliqué au développement de Wavve 6

2.1 Diagramme de cas d'utilisation global de l'application Wavve 12

3.1 Diagramme de séquences "Login" 14
3.2 Diagramme de séquences "Lancement de musique" 14
3.3 Diagramme de séquences "Création de Playlist" 15
3.4 Diagramme de classes de l'application Wavve 15
3.5 Modèle relationnel de la base de données 16
3.6 Architecture logicielle de l'application Wavve 17

4.1 Interface utilisateur - Page d'accueil 19
4.2 Interface de création de compte 20
4.3 Interface de connexion 20
4.4 Interface de création d'une playlist 21
4.5 Interface de navigation musicale 24
4.6 Interface de gestion des playlists 24
4.7 Interface de profil utilisateur 25

Liste des tableaux

1.1 Planning prévisionnel de développement de l'application 7

2.1 Description du cas d'utilisation « Création de playlist » 11

Introduction générale

Dans un paysage numérique où la consommation musicale est devenue omniprésente, l'accès à un catalogue musical diversifié et personnalisé représente un enjeu majeur pour les mélomanes. Face à cette demande croissante, notre équipe a conçu **Wavve**, une application de streaming musical intuitive et performante.

Wavve se positionne comme une alternative innovante dans le domaine du streaming audio, en proposant une expérience utilisateur fluide et des fonctionnalités comparables à celles des leaders du marché tels que Spotify. Notre plateforme permet aux utilisateurs de découvrir, écouter et partager leur musique préférée, tout en bénéficiant de recommandations personnalisées basées sur leurs habitudes d'écoute.

L'application a été développée en utilisant un stack technologique robuste et moderne. Le backend repose sur **Python**, choisi pour sa polyvalence et sa riche bibliothèque d'outils de traitement audio et d'analyse de données. La persistance des données est assurée par une base de données **MySQL**, offrant performance et fiabilité pour la gestion d'un large catalogue musical et des profils utilisateurs. L'interface utilisateur, élégante et responsive, a été conçue avec **HTML5**, **CSS3** et **JavaScript**, garantissant une expérience cohérente sur tous les appareils.

Parmi les fonctionnalités clés de Wavve, on retrouve :

— La création de comptes utilisateurs personnalisés
— L'exploration d'un vaste catalogue musical catégorisé par genres, artistes et albums
— La création et le partage de playlists personnalisées
— Un système de recommandation basé sur les préférences utilisateur
— Un lecteur audio intuitif avec contrôle de volume, lecture aléatoire et répétition
— Une interface de recherche avancée
— Des fonctionnalités sociales permettant de suivre d'autres utilisateurs et découvrir leurs playlists

Ce projet s'inscrit dans notre volonté d'exploiter les technologies modernes de développement web et de gestion de données pour créer une solution de streaming musical accessible et performante. L'architecture client-serveur que nous avons adoptée permet une évolutivité optimale, garantissant la possibilité d'enrichir l'application avec de nouvelles fonctionnalités au fil du temps.

Wavve représente non seulement une réponse technique aux défis du streaming musical contemporain, mais également une vision où la technologie se met au service de l'expérience musicale, rendant la musique plus accessible, plus personnelle et plus sociale.

Chapitre 1

Présentation du cadre de projet

1.1 Introduction

Le streaming musical actuel, malgré sa popularité, présente encore des limites en termes d'expérience utilisateur, avec des interfaces complexes et des recommandations souvent inadaptées aux préférences individuelles des auditeurs.

Wavve répond à ce défi en proposant une plateforme de streaming musicale intuitive, développée avec Python, MySQL et des technologies web modernes (HTML, CSS, JavaScript). Notre application offre une alternative aux services existants en mettant l'accent sur la personnalisation et la facilité d'utilisation.

Notre objectif est simple : permettre aux utilisateurs de découvrir, organiser et partager leur musique de manière plus fluide et personnalisée, transformant ainsi leur expérience d'écoute quotidienne.

1.2 Contexte et état de l'art

1.2.1 Présentation des plateformes existantes

Le marché du streaming musical est dominé par plusieurs acteurs majeurs comme Spotify, Apple Music, Deezer et YouTube Music. Ces plateformes offrent des catalogues musicaux vastes et des fonctionnalités avancées.

1.2.2 Limites des solutions actuelles

Aujourd'hui, malgré la popularité des plateformes de streaming musical, il reste difficile pour certains utilisateurs d'accéder à une expérience personnalisée et fluide sans passer par des services payants ou complexes. De plus, de nombreuses applications manquent de simplicité, d'ergonomie ou de fonctionnalités communautaires telles que le partage ou la recommandation musicale.

1.2.3 Problématique

La problématique centrale de notre projet réside dans la nécessité de créer une alternative accessible et personnalisée aux plateformes de streaming existantes, en mettant l'accent sur l'expérience utilisateur et la simplicité d'utilisation.

1.3 Solution proposée

Pour répondre à cette problématique, nous avons développé **Wavve**, une application web de streaming musical inspirée de Spotify. Elle permet aux utilisateurs d'écouter de la musique, de créer des playlists, de rechercher des artistes ou des albums, et de suivre d'autres utilisateurs. L'interface est simple, moderne et accessible, et l'application intègre un système de recommandations automatiques pour enrichir l'expérience musicale.

1.4 Choix de modèle de développement

Pour développer notre application, nous avons opté pour le **modèle en V**. Ce modèle offre une structure claire avec des phases de conception suivies immédiatement par des phases de tests, ce qui garantit une validation continue. Il convient parfaitement à notre projet car les besoins ont été bien définis dès le début et ont peu évolué. Ce choix nous a permis de limiter les erreurs, de mieux organiser notre travail, et de livrer une application stable et fonctionnelle, en accord avec les principes du cours de Génie Logiciel.

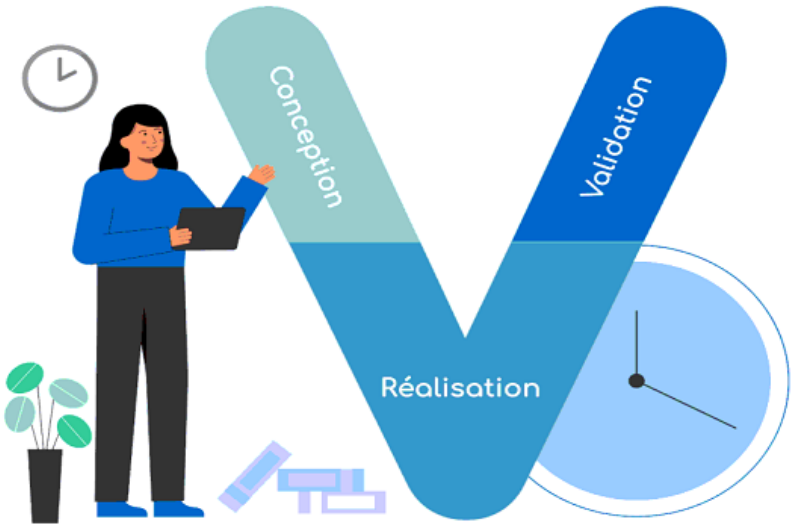


FIGURE 1.1 – Le cycle en V appliqué au développement de Wavve

1.5 Planning prévisionnel

1.6 Conclusion

Ce projet nous a permis de développer une application web de musique nommée **Wavve**. Nous avons pu mettre en œuvre nos compétences en développement web, base de données et conception d'interfaces. Ce travail a été une expérience enrichissante qui nous a permis de consolider nos acquis et de découvrir de nouveaux aspects du développement d'applications.

TABLE 1.1 – Planning prévisionnel de développement de l'application

Étape	**Mars**				**Avril**				**Mai**			
	1	**2**	**3**	**4**	**1**	**2**	**3**	**4**	**1**	**2**	**3**	**4**
Étude préalable			x	x								
Conception					x	x						
Réalisation							x	x	x			
Test et Validation										x	x	x

Chapitre 2

Spécification des besoins

2.1 Introduction

Dans ce chapitre, nous allons détailler les besoins auxquels doit répondre l'application Wavve. Ces besoins sont divisés en deux catégories : les besoins fonctionnels, qui décrivent ce que l'application doit faire, et les besoins non fonctionnels, qui définissent les contraintes techniques, de performance et d'ergonomie. Ce chapitre comprend également les cas d'utilisation permettant de visualiser l'interaction entre les utilisateurs et le système.

2.2 Spécification des besoins fonctionnels

2.2.1 Gestion des utilisateurs

— Inscription et authentification des utilisateurs via un formulaire sécurisé.
— Création de profils utilisateurs contenant les informations de base (nom, e-mail, photo, préférences musicales).
— Suivi d'autres utilisateurs (fonction de « follow »).
— Historique d'écoute consultable par l'utilisateur.

2.2.2 Gestion de la musique (artistes, albums, titres)

— Consultation des artistes et de leurs albums.
— Affichage des titres disponibles dans chaque album.
— Possibilité de lire un morceau via un lecteur intégré.
— Récupération des données musicales à partir de l'API Deezer.

2.2.3 Création et gestion des playlists

— Création de playlists personnelles par l'utilisateur.
— Ajout et suppression de morceaux dans une playlist.
— Partage de playlists avec d'autres utilisateurs.
— Affichage des playlists populaires ou recommandées.

2.2.4 Système de recherche et recommandations

— Barre de recherche permettant de retrouver des morceaux, artistes ou albums.

— Suggestions dynamiques basées sur l'historique d'écoute.
— Recommandations personnalisées de playlists.

2.3 Spécification des besoins non fonctionnels

Les besoins non fonctionnels définissent les exigences de qualité que doit respecter l'application Wavve. Ils ne concernent pas directement les fonctionnalités offertes, mais influencent fortement l'expérience utilisateur, la maintenabilité du système, et sa robustesse. Voici les principaux besoins identifiés :

2.3.1 Interface utilisateur responsive et moderne

L'interface de Wavve a été conçue pour s'adapter automatiquement aux différentes tailles d'écran (ordinateurs, tablettes, smartphones), grâce à l'utilisation de HTML5, CSS3 et de techniques de responsive design. Cela garantit une navigation fluide, que ce soit sur un grand écran ou un appareil mobile. L'ergonomie a été soigneusement travaillée pour offrir une expérience intuitive et agréable.

2.3.2 Compatibilité multiplateforme et navigateurs

L'application doit être pleinement fonctionnelle sur les principaux navigateurs modernes tels que Google Chrome, Mozilla Firefox, Microsoft Edge et Safari. Le code a été testé pour garantir l'uniformité de l'affichage et du comportement des interfaces, quelle que soit la plateforme utilisée (Windows, macOS, Linux).

2.3.3 Performances et optimisation du temps de chargement

L'optimisation des performances est un critère fondamental pour garantir la fluidité de l'écoute et la réactivité de l'interface. Le code JavaScript est structuré de manière asynchrone pour éviter les blocages, et les requêtes SQL sont optimisées pour réduire le temps de réponse. Les fichiers CSS et JS sont minifiés, et les ressources médias sont chargées de manière progressive.

2.3.4 Sécurité des données utilisateurs

La sécurité des utilisateurs est une priorité. Les mots de passe sont hachés (et non stockés en clair) à l'aide d'un algorithme sécurisé (comme bcrypt). Les sessions utilisateurs sont protégées contre les attaques CSRF et XSS. Les pages sensibles sont accessibles uniquement après authentification. L'application respecte également les bonnes pratiques en matière de gestion des données personnelles.

2.3.5 Architecture modulaire et maintenabilité

Le code côté serveur est structuré de façon modulaire, avec une séparation claire entre les modèles, les vues, les routes et les contrôleurs (selon le principe MVC). Cette organisation facilite la lecture, les tests et les évolutions futures. Le choix d'une base de données relationnelle (MySQL) assure une structuration robuste des données et facilite leur manipulation via des requêtes SQL efficaces.

2.3.6 Extensibilité et évolutivité

Wavve a été pensée pour être évolutive : de nouvelles fonctionnalités (comme la diffusion en direct, l'ajout de commentaires ou l'intégration d'autres APIs musicales) pourront être ajoutées sans nécessiter une refonte complète du système. Le découpage logique du backend et du frontend permet une évolution séparée de chaque couche.

2.4 Cas d'utilisation

2.4.1 Acteurs du système

L'application de musique Wavve est conçue pour un seul type d'acteur principal : l'utilisateur (ou client). Cet acteur représente toute personne utilisant la plateforme pour accéder à du contenu musical en ligne.

Client : L'utilisateur est au centre de l'expérience proposée par Wavve. Il peut créer un compte, se connecter à la plateforme et accéder à son espace personnel. Il a la possibilité de rechercher de la musique, de consulter les artistes, albums et titres disponibles, ainsi que de créer et gérer ses propres playlists. L'interface lui permet également d'écouter des morceaux en streaming et de bénéficier de suggestions musicales basées sur ses préférences ou son historique d'écoute.

Les cas d'utilisation présentés dans cette section décrivent les différentes actions que l'utilisateur peut effectuer au sein de l'application. Ils permettent d'avoir une vision globale et structurée des principales fonctionnalités offertes par Wavve.

2.4.2 Description des cas d'utilisation

Dans cette section, nous allons décrire les cas d'utilisation de l'application Wavve, centrés sur l'unique acteur du système : l'utilisateur (client). Chaque cas représente une interaction typique entre l'utilisateur et l'application. Voici un exemple structuré :

2.4.3 Diagramme de cas d'utilisation global

TABLE 2.1 – Description du cas d'utilisation « Création de playlist »

Cas n°	1
Acteur(s)	Utilisateur (Client)
Objectif	Permettre à l'utilisateur de créer une nouvelle playlist musicale personnelle.
Pré-condition(s)	L'utilisateur doit être connecté à son compte sur l'application Wavve.
Post-condition(s)	Une nouvelle playlist est créée et associée au profil de l'utilisateur.
Scénario nominal	1. L'utilisateur se connecte à l'application. 2. Il accède à la section « Mes Playlists ». 3. Il clique sur « Créer une playlist ». 4. Il saisit le nom de la playlist. 5. Il valide la création. 6. La playlist est enregistrée dans la base de données et visible dans l'espace personnel de l'utilisateur.
Scénario alternatif	Si l'utilisateur tente de créer une playlist sans nom, l'application affiche un message d'erreur l'invitant à saisir un nom valide.

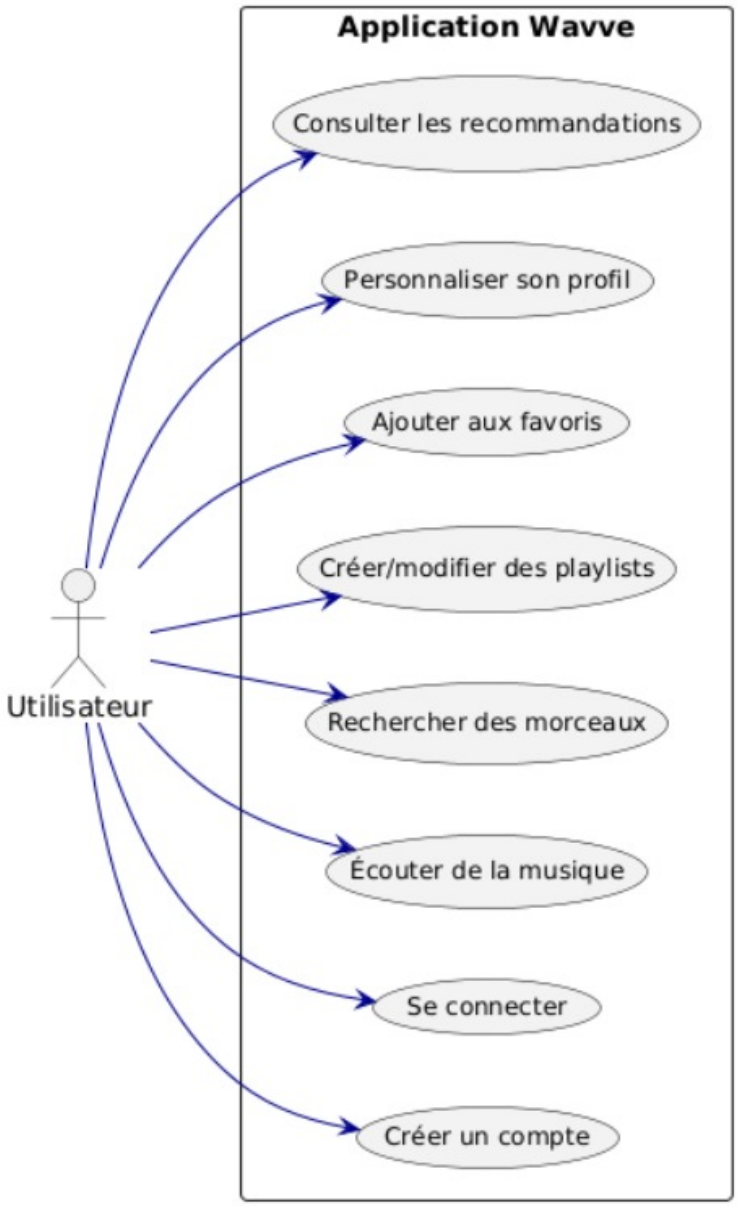


FIGURE 2.1 – Diagramme de cas d'utilisation global de l'application Wavve

Chapitre 3

Conception du système

3.1 Introduction

Dans ce chapitre, nous allons modéliser notre application de streaming musical selon deux aspects fondamentaux : statique et dynamique.

1. La **modélisation statique** décrit la structure du système (classes, déploiement, relations).
2. La **modélisation dynamique** représente les interactions et les comportements du système au fil du temps (séquences, collaboration, états).

3.2 Modélisation dynamique

3.2.1 Diagrammes de séquences

Un diagramme de séquence est un diagramme d'interaction dont le but est de décrire comment les objets collaborent au cours du temps et quelles responsabilités ils assument. Il décrit un scénario d'un cas d'utilisation.

Avec les intéressants ajouts au diagramme de séquences apportés par UML 2, en particulier les cadres d'interactions (avec les opérateurs *loop*, *opt* et *alt* par exemple), ainsi que la possibilité de référencer une interaction décrite par ailleurs, le diagramme de séquence système nous semble constituer une excellente solution.

Les différents messages représentés par un diagramme de séquence sont :

— **Message asynchrone :** Il n'attend pas de réponse et ne bloque pas l'émetteur qui ne sait pas si le message arrivera à destination.

— **Message synchrone :** L'émetteur reste alors bloqué le temps que dure l'invocation de l'opération.

— **Messages de création et destruction d'instance :** La création d'un objet est matérialisée par une flèche qui pointe sur le sommet d'une ligne de vie et la destruction d'un objet est matérialisée par une croix qui marque la fin de la ligne de vie de l'objet.

3.2.2 Diagrammes de collaboration

Les diagrammes de collaboration montrent les interactions entre objets dans un contexte particulier. Ils mettent l'accent sur l'organisation structurelle des objets qui envoient et

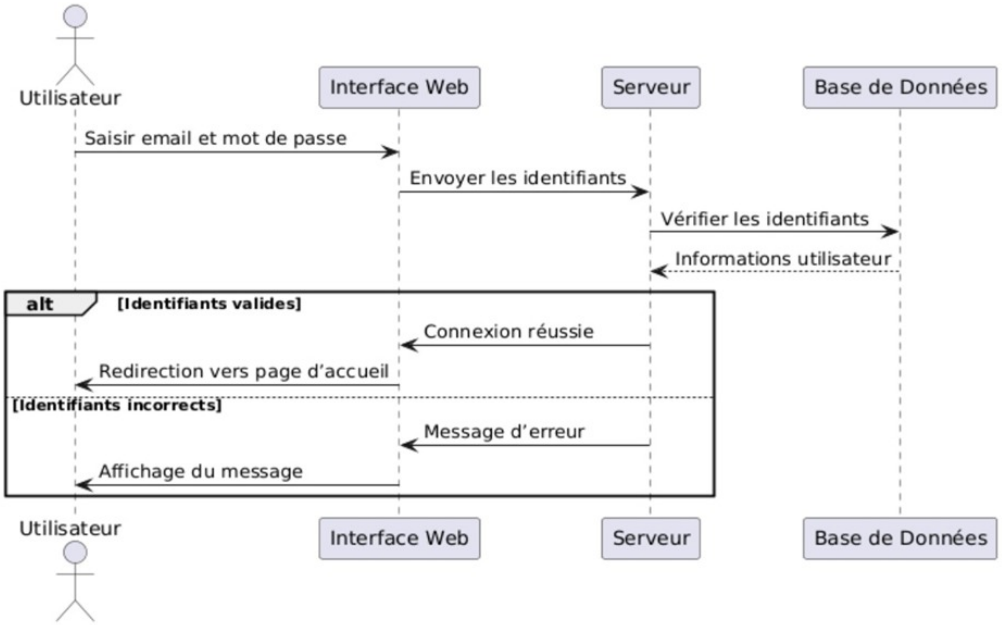


FIGURE 3.1 – Diagramme de séquences "Login"

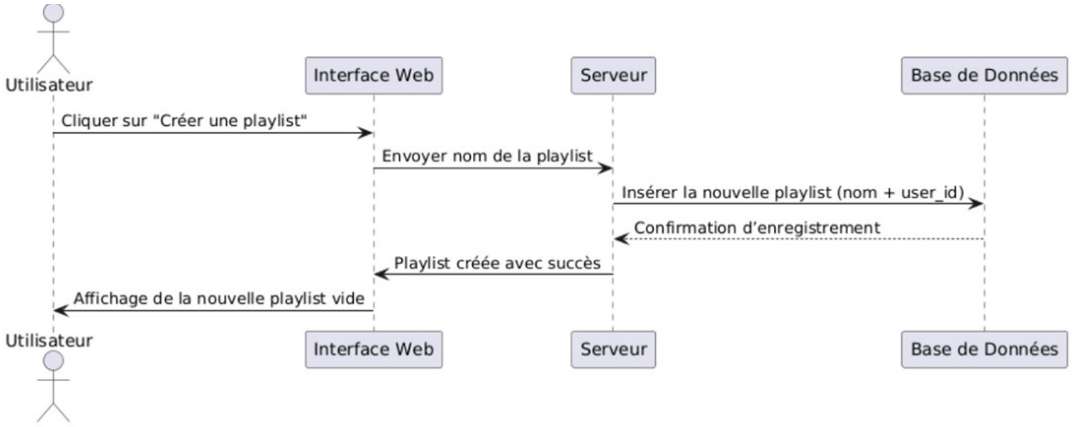


FIGURE 3.2 – Diagramme de séquences "Lancement de musique"

reçoivent des messages.

3.2.3 Diagrammes d'états

Les diagrammes d'états décrivent les différents états d'un objet au cours de sa vie et les transitions entre ces états en réponse à des événements.

3.2.4 Diagrammes d'état-transition

Ces diagrammes permettent de modéliser le comportement dynamique d'une classe en montrant les états possibles des instances de cette classe et les transitions entre ces états.

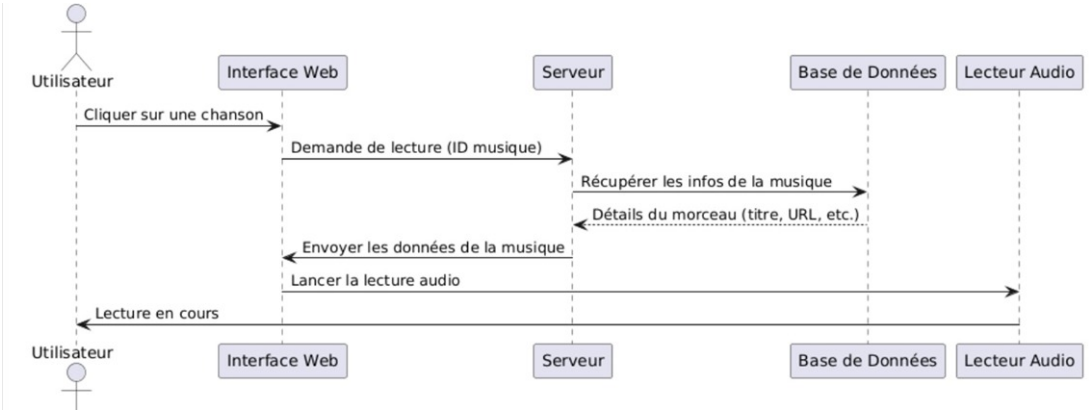


FIGURE 3.3 – Diagramme de séquences "Création de Playlist"

3.2.5 Diagrammes d'activité

Les diagrammes d'activité représentent les flux de contrôle et de données dans un système. Ils permettent de modéliser les processus métier et les algorithmes.

3.3 Modélisation statique

3.3.1 Diagramme de classes

Le diagramme de classes présente la structure statique du système en montrant les classes, leurs attributs, leurs méthodes et les relations entre elles.

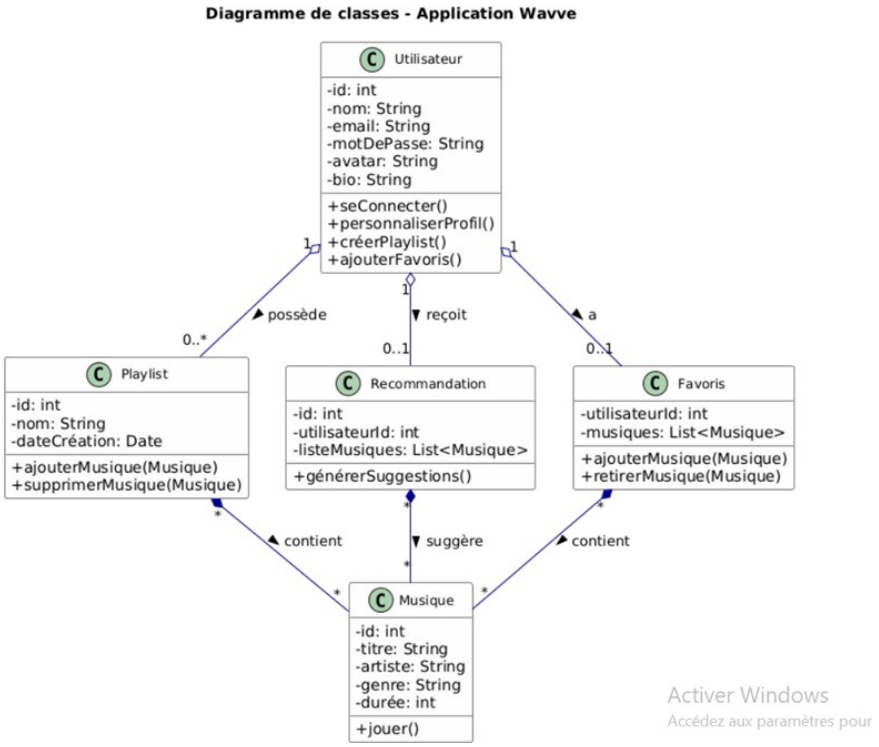


FIGURE 3.4 – Diagramme de classes de l'application Wavve

3.3.2 Modèle relationnel de la base de données

Le modèle relationnel découle du diagramme de classes en appliquant les règles de passage du modèle objet vers le modèle relationnel.

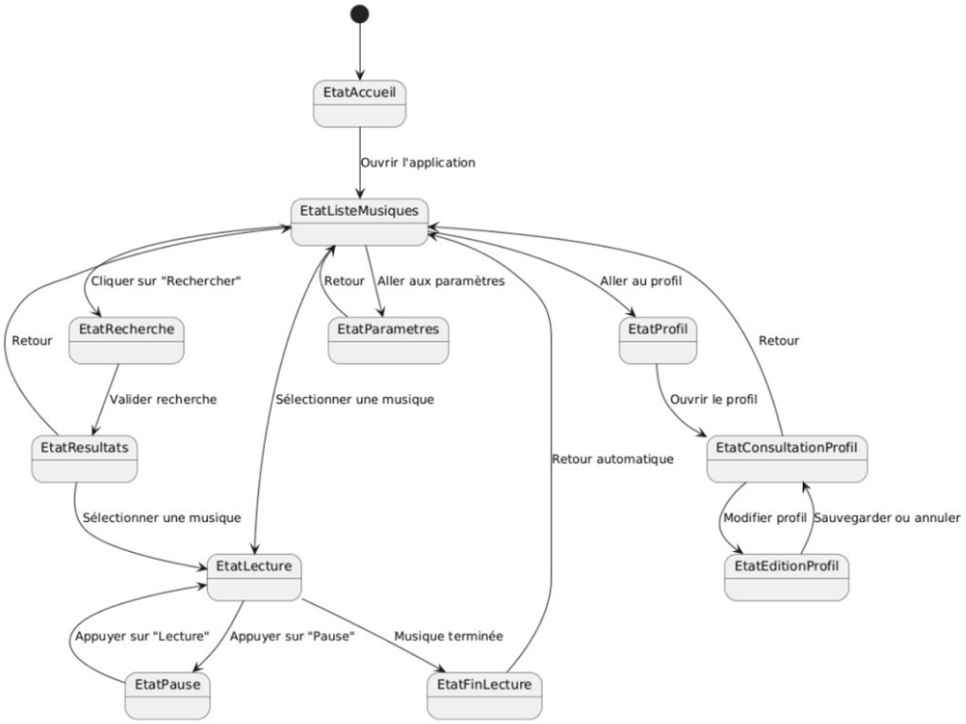


FIGURE 3.5 – Modèle relationnel de la base de données

3.3.3 Architecture de l'application

Architecture logicielle

Cette section présente le diagramme de composants de l'application Wavve. Ce diagramme illustre les différentes parties logicielles qui composent l'application (frontend, backend, base de données, API Deezer, etc.), ainsi que leurs interactions. Il permet de visualiser l'organisation modulaire du code, facilitant la maintenance, l'évolutivité et la réutilisabilité des composants.

Architecture matérielle

Dans cette partie, nous présentons le diagramme de déploiement, qui décrit la structure matérielle sur laquelle l'application est exécutée. Ce diagramme met en évidence les nœuds matériels (serveur web, base de données, client) ainsi que les connexions réseau entre eux. Il reflète la configuration réelle ou prévue de l'environnement de production ou de test.

Remarques importantes :

— Les nœuds (serveur, client, base de données) sont représentés de manière explicite.
— Les liens de communication (HTTP, SQL, API) doivent être indiqués entre les nœuds pour représenter le flux d'échange.

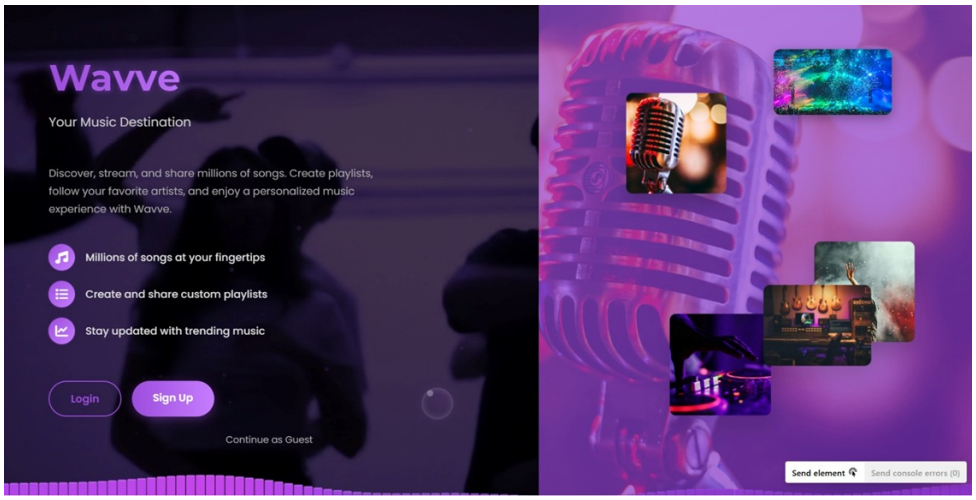


FIGURE 3.6 – Architecture logicielle de l'application Wavve

3.4 Conclusion

Ce chapitre a permis de présenter la conception technique du système, à travers des diagrammes dynamiques (séquences, états), statiques (classes, base de données), ainsi que les architectures logicielle et matérielle. Ces éléments offrent une vue d'ensemble claire et structurée de la manière dont l'application Wavve a été pensée et organisée avant sa mise en œuvre.

Chapitre 4

Réalisation du système

4.1 Introduction

Dans ce chapitre, nous présentons la phase de développement de l'application Wavve. Après avoir défini les besoins et conçu l'architecture du système, nous sommes passés à la mise en œuvre technique. Cette étape comprend le choix des outils, l'installation de l'environnement de développement, le codage des fonctionnalités principales et la création des interfaces graphiques.

4.2 Environnement de développement

4.2.1 Environnement matériel

Le développement de l'application a été réalisé sur un environnement matériel standard, composé de :

— **Processeur** : Intel Core i7
— **Mémoire RAM** : 16 Go
— **Disque** : SSD 256 Go
— **Système d'exploitation** : Windows 11

Ces caractéristiques ont été suffisantes pour assurer le bon fonctionnement des outils de développement et d'exécution nécessaires au projet.

4.2.2 Environnement logiciel

Pour mener à bien ce projet, nous avons utilisé un ensemble d'outils modernes adaptés au développement web full stack :

Frontend :
— **Langage** : JavaScript (React JS), HTML
— **Framework CSS** : Tailwind CSS

Backend :
— **Langage** : Python
— **Framework** : Django

Base de données :
— **SGBD** : SQLite

Outils de développement :

— **IDE** : Visual Studio Code
— **Outil de modélisation** : StarUML

Ces outils ont été choisis pour leur compatibilité, leur large communauté, et leur capacité à produire un site web interactif, performant et responsive.

4.3 Principales interfaces graphiques

L'application Wavve dispose d'une interface utilisateur moderne, intuitive et responsive, pensée pour offrir une expérience fluide sur tout type de support (ordinateur, tablette, smartphone). Voici les principales interfaces graphiques développées :

4.3.1 Page d'accueil

Cette interface constitue la porte d'entrée de l'application Wavve. Elle présente brièvement le concept de la plateforme et met en avant les fonctionnalités phares telles que l'écoute de musique en streaming, la création de playlists et les recommandations personnalisées. Elle propose également un accès rapide à l'inscription et à la connexion.

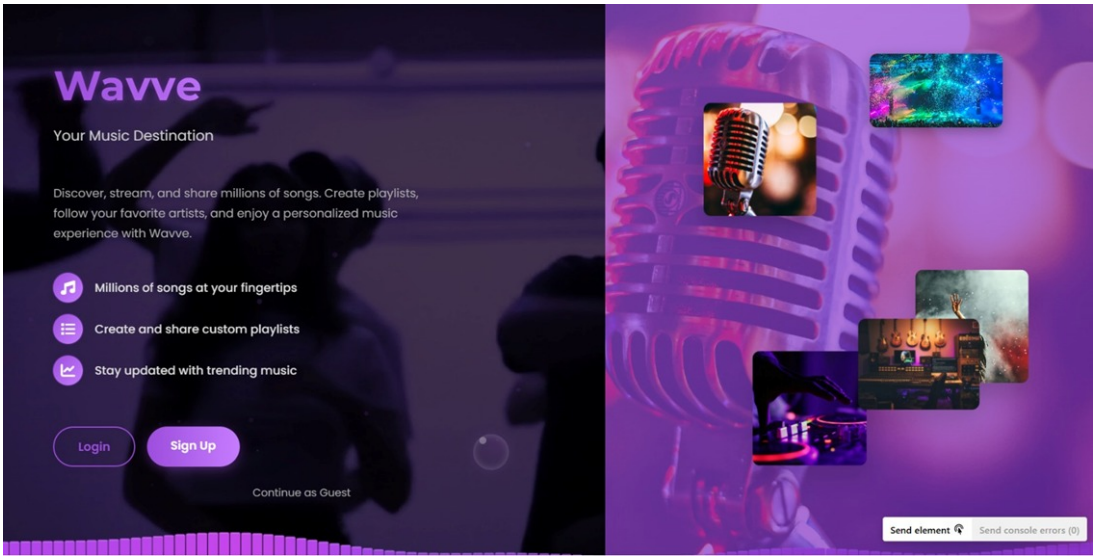


FIGURE 4.1 – Interface utilisateur - Page d'accueil

4.3.2 Page de création de compte

Elle permet à un nouvel utilisateur de s'inscrire sur la plateforme en remplissant un formulaire contenant les informations suivantes : nom, adresse email, mot de passe. Une vérification des champs est mise en place pour assurer la validité des données saisies (mot de passe sécurisé, email au bon format, etc.).

4.3.3 Page de connexion

L'utilisateur enregistré peut accéder à son espace personnel en saisissant son email et mot de passe. En cas d'échec, des messages d'erreur clairs sont affichés.

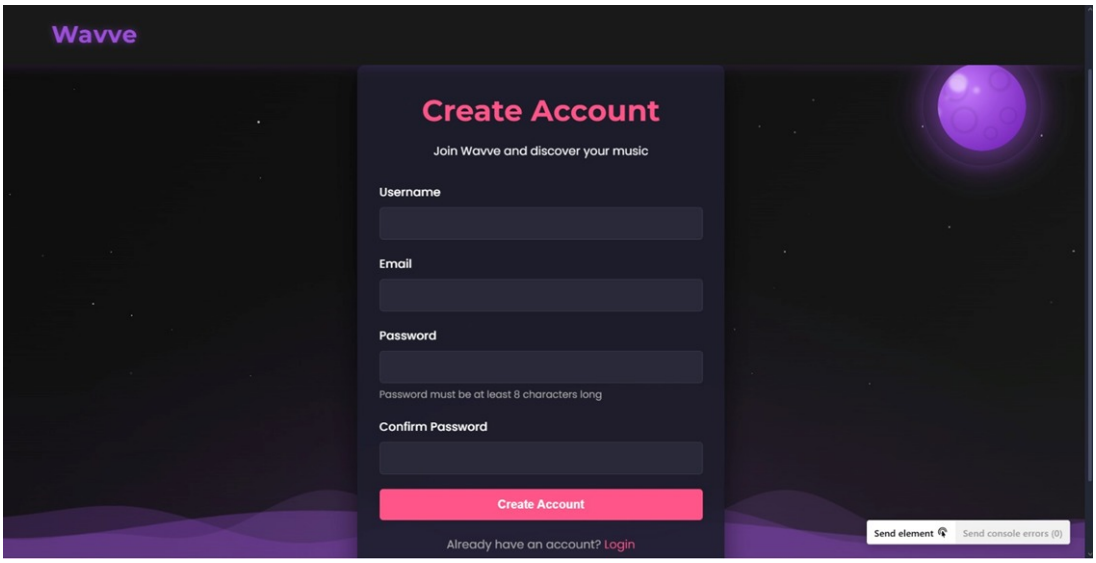


FIGURE 4.2 – Interface de création de compte

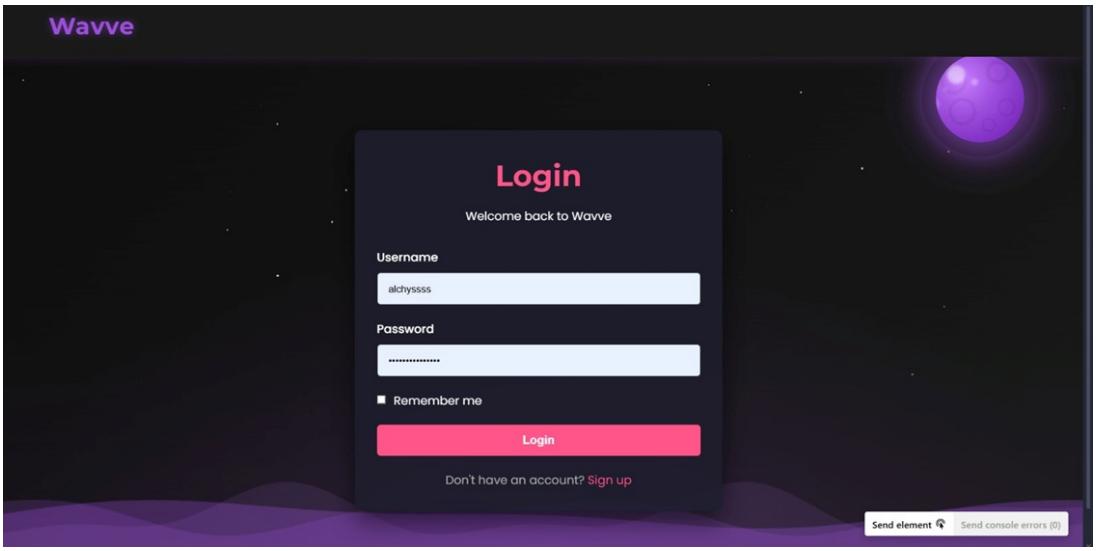


FIGURE 4.3 – Interface de connexion

4.3.4 Formulaire de création de playlist

Cette interface permet à l'utilisateur de créer une nouvelle playlist musicale. Il lui suffit de renseigner un nom pour la playlist, puis d'y ajouter des morceaux via une recherche intégrée. L'interface permet également de modifier ou supprimer des playlists existantes.

4.3.5 Interface de lecture musicale

Le lecteur audio intégré offre toutes les fonctionnalités essentielles : lecture/pause, contrôle du volume, avance/retour, lecture aléatoire et répétition. L'interface affiche également les informations du morceau en cours (titre, artiste, album, pochette).

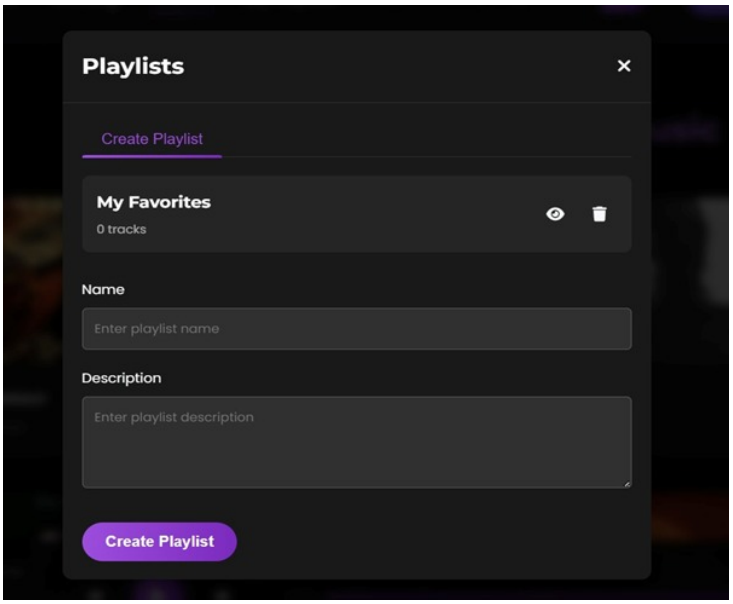


FIGURE 4.4 – Interface de création d'une playlist

4.3.6 Interface de recherche

La barre de recherche permet aux utilisateurs de trouver rapidement des artistes, albums ou morceaux. Les résultats sont affichés de manière organisée avec des suggestions de recherche automatiques.

4.4 Conclusion

La phase de réalisation nous a permis de transformer la conception théorique de Wavve en une application web fonctionnelle. Grâce à une bonne planification et à l'utilisation d'outils appropriés, nous avons pu développer une plateforme complète qui répond aux besoins identifiés. L'architecture modulaire du projet permet une bonne maintenabilité et une facilité d'évolution pour des fonctionnalités futures.

Conclusion générale

Le développement de l'application de streaming musical Wavve nous a permis de mettre en pratique l'ensemble des compétences acquises tout au long de notre formation. Inspirée des grandes plateformes comme Spotify, Wavve intègre les fonctionnalités essentielles d'une application musicale moderne : navigation par artistes et albums, création de playlists, recherche intelligente, recommandations personnalisées et une interface utilisateur responsive.

Nous avons pu travailler sur toutes les étapes de conception d'une application web : analyse des besoins, modélisation, choix technologiques, développement et tests. Ce projet nous a permis de renforcer nos connaissances en Python, MySQL, HTML/CSS/JS et d'adopter une démarche de développement structurée basée sur le modèle en V.

L'application développée répond aux objectifs fixés initialement :

— Offrir une interface utilisateur intuitive et moderne
— Permettre la gestion complète des playlists personnalisées
— Intégrer un système de recommandations musicales
— Assurer une expérience d'écoute fluide et responsive
— Maintenir un haut niveau de sécurité pour les données utilisateurs

Les défis techniques rencontrés nous ont permis d'approfondir nos compétences en développement web full-stack, notamment dans l'intégration d'APIs externes, la gestion de bases de données relationnelles et l'optimisation des performances front-end.

En termes de perspectives d'évolution, plusieurs améliorations pourraient être apportées à Wavve :

— Ajout d'un mode hors ligne pour l'écoute sans connexion
— Intégration de fonctionnalités sociales plus avancées
— Développement d'une application mobile native
— Amélioration des algorithmes de recommandation avec l'intelligence artificielle
— Intégration de nouvelles sources musicales via d'autres APIs

En résumé, ce projet a été une expérience enrichissante, aussi bien sur le plan technique que personnel. Il nous a permis de mieux comprendre les enjeux de la création d'une plateforme web interactive et scalable, et nous ouvre des perspectives intéressantes pour des évolutions futures dans le domaine du développement d'applications web.

Bibliographie et Nétographie

Bibliographie

1. BUREL, Guy. *Développement d'applications web avec Django*. Éditions ENI, 2022.
2. LEMAY, Laura. *Apprenez à programmer avec Python*. Eyrolles, 2021.
3. ROQUES, Pascal. *UML 2 par la pratique*. Eyrolles, 5e édition, 2010.
4. MEYER, Bertrand. *Ingénierie logicielle*. Dunod, 2019.

Nétographie

— `https://developer.deezer.com/` – Documentation officielle de l'API Deezer
— `https://flask.palletsprojects.com/` – Documentation du framework Flask
— `https://getbootstrap.com/` – Documentation Bootstrap
— `https://www.w3schools.com/` – Références HTML/CSS/JavaScript
— `https://dev.mysql.com/doc/` – Documentation MySQL
— `https://reactjs.org/` – Documentation officielle de React
— `https://tailwindcss.com/` – Documentation Tailwind CSS
— `https://docs.djangoproject.com/` – Documentation Django

ANNEXES

ANNEXE A : Captures d'écran supplémentaires

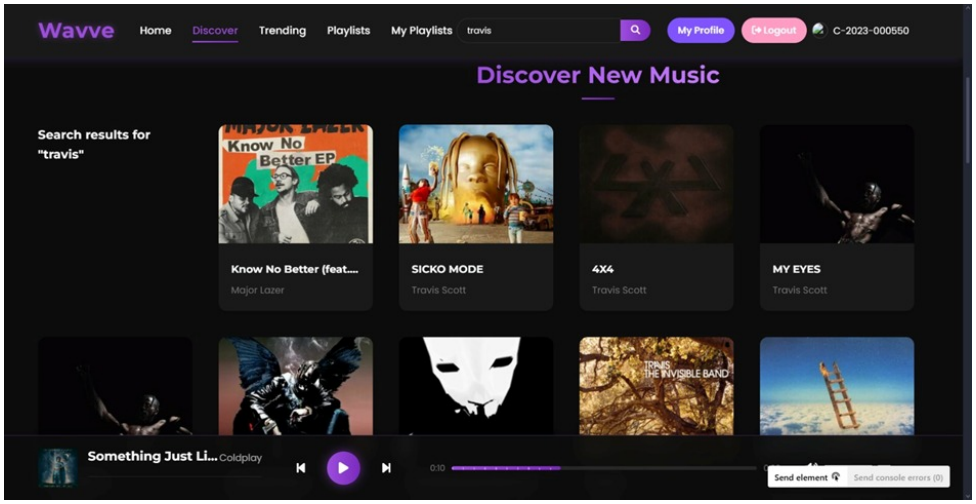


FIGURE 4.5 – Interface de navigation musicale

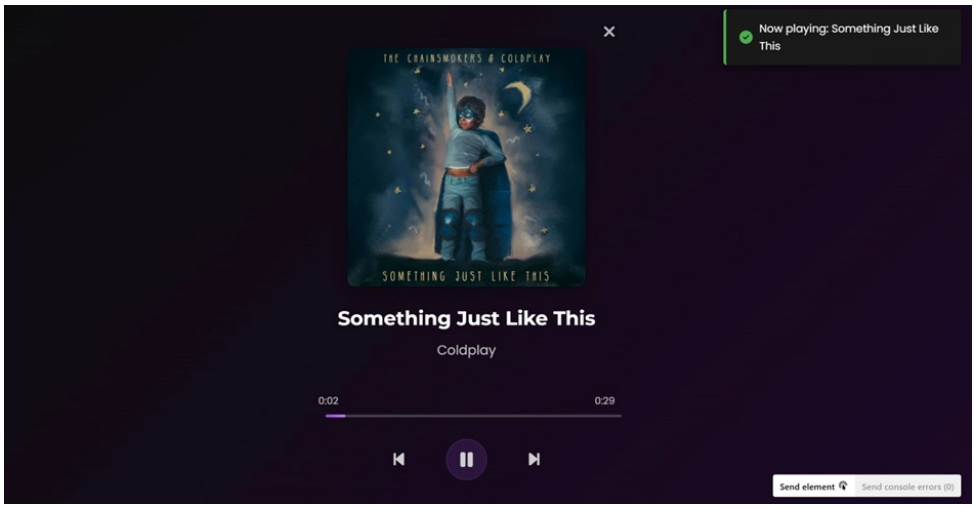


FIGURE 4.6 – Interface de gestion des playlists

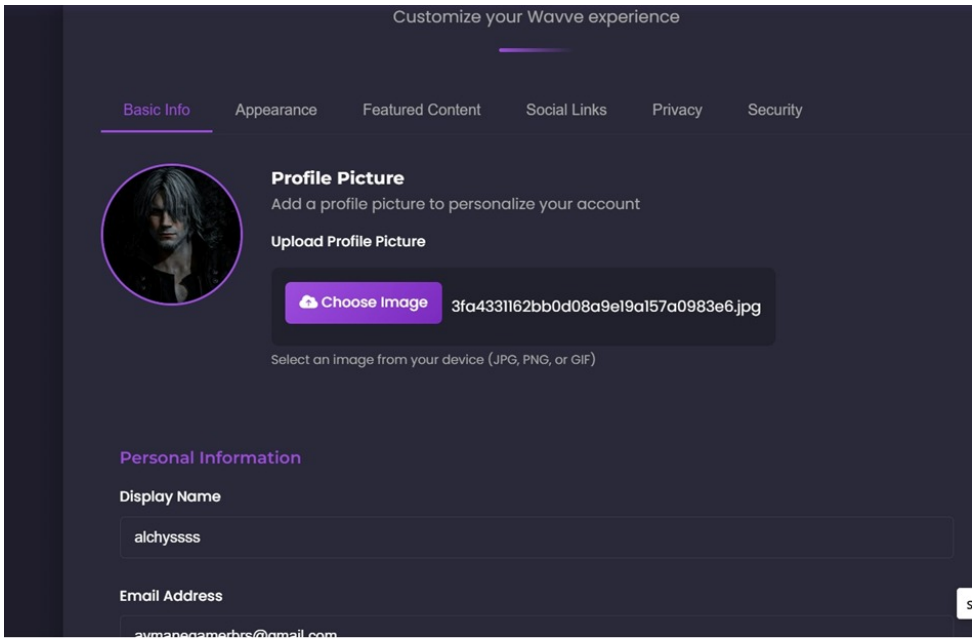


FIGURE 4.7 – Interface de profil utilisateur

ANNEXE B : Extraits de code ou scripts utiles

Extrait de code Python - Modèle Utilisateur (Django)

```
from django.contrib.auth.models import AbstractUser
from django.db import models

class User(AbstractUser):
    email = models.EmailField(unique=True)
    date_of_birth = models.DateField(null=True, blank=True)
    profile_picture = models.ImageField(
        upload_to='profile_pics/',
        null=True,
        blank=True
    )

    def __str__(self):
        return self.username

class Playlist(models.Model):
    name = models.CharField(max_length=100)
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"{self.name} - {self.user.username}"
```

Extrait de code JavaScript - Lecteur Audio

```
class AudioPlayer {
    constructor() {
        this.audio = new Audio();
        this.isPlaying = false;
        this.currentTrack = null;
    }

    play(trackUrl) {
        this.audio.src = trackUrl;
        this.audio.play();
        this.isPlaying = true;
        this.updateUI();
    }

    pause() {
        this.audio.pause();
        this.isPlaying = false;
        this.updateUI();
    }

    updateUI() {
        const playButton = document.getElementById('play-button');
        playButton.textContent = this.isPlaying ? 'Pause' : 'Play';
    }
}
```

ANNEXE C : Recommandations et perspectives

Voici quelques pistes d'amélioration et d'extension possibles pour faire évoluer l'application Wavve à l'avenir :

Améliorations techniques

- **Mode hors ligne** : Permettre aux utilisateurs de télécharger leurs morceaux favoris pour une écoute sans connexion Internet.
- **Optimisation des performances** : Implémenter un système de cache plus sophistiqué et optimiser les requêtes de base de données.
- **Architecture microservices** : Diviser l'application en microservices pour améliorer la scalabilité.
- **Tests automatisés** : Développer une suite de tests unitaires et d'intégration complète.

Nouvelles fonctionnalités

- **Système de commentaires** : Permettre aux utilisateurs de commenter et évaluer les playlists et morceaux.
- **Recommandations IA** : Intégrer des algorithmes d'apprentissage automatique pour des recommandations plus précises.

— **Fonctionnalités sociales** : Ajouter un système de messages privés et de partage sur les réseaux sociaux.
— **Statistiques avancées** : Fournir des analyses détaillées des habitudes d'écoute des utilisateurs.

Extensions de plateforme

— **Application mobile native** : Développer des applications iOS et Android natives.
— **Application desktop** : Créer une application de bureau avec Electron.
— **Intégration APIs multiples** : Connecter Spotify, YouTube Music, Apple Music.
— **Support vocal** : Intégrer des commandes vocales pour une navigation mains libres.

Considérations de déploiement

— **Cloud hosting** : Déployer sur des plateformes comme AWS, Google Cloud ou Heroku.
— **CDN** : Utiliser un réseau de diffusion de contenu pour optimiser les temps de chargement.
— **Monitoring** : Implémenter des outils de surveillance des performances et d'alertes.
— **Sécurité avancée** : Ajouter l'authentification à deux facteurs et le chiffrement des données.

Ces recommandations permettraient de transformer Wavve en une plateforme de streaming musical complète et compétitive sur le marché actuel.